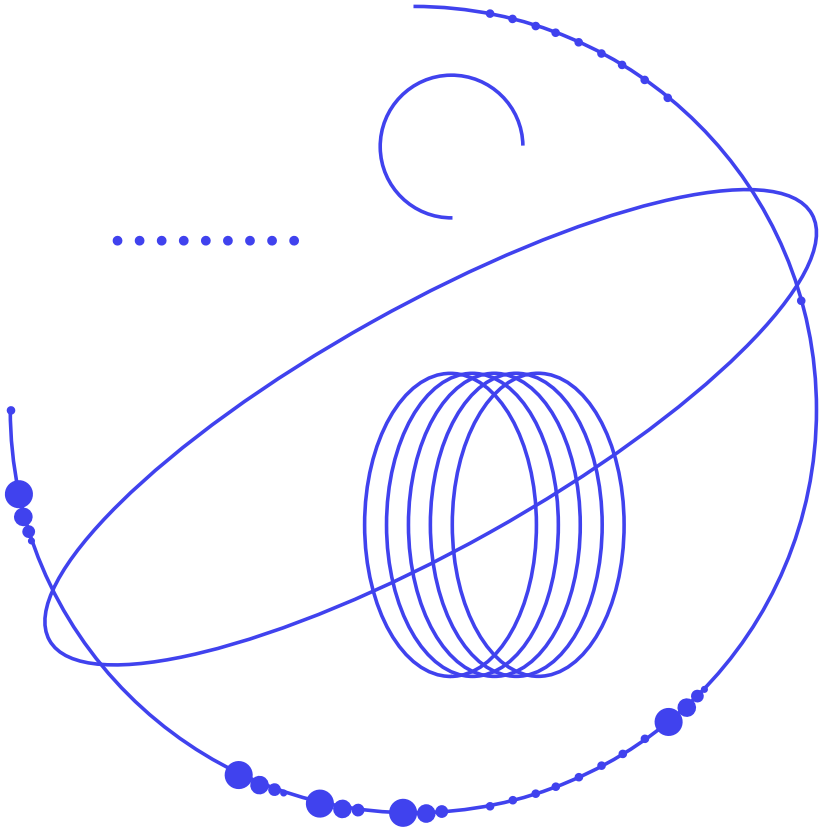
Школа анализа данных

Lecture 5.1

Production Engineering for LLM Agents

Daniil Artamonov
Head of AI Platform and Tools Development for IT

ACT 1



Prototype: 7 Problems

FlightAgent: Starting Point

FlightAgent

LLM (brain)	Tools (hands)	Loop (spine)
GPT-4 / Claude Generates responses and decisions	search_flights() book_ticket() get_weather()	Observe → Think → Act → Observe until task done

Works beautifully in Jupyter. Demo went great, everyone clapped

Ready for production? Not yet

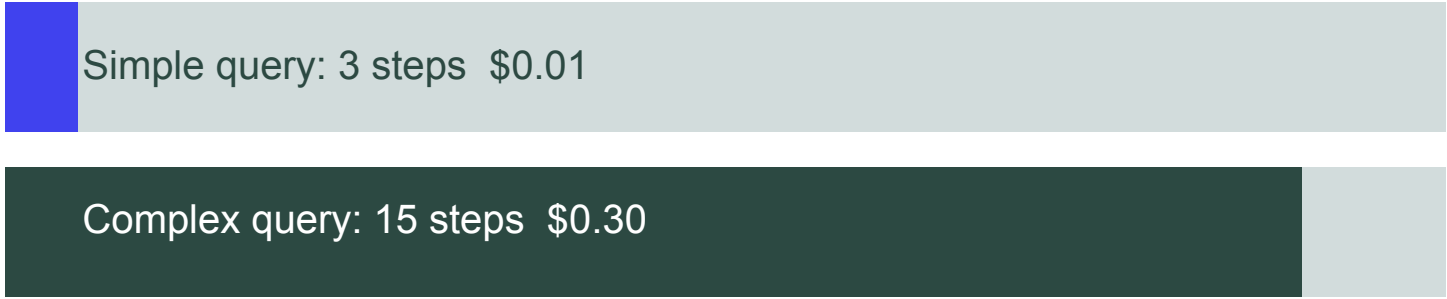
Problem: Slow

5–10s

per request
(3 LLM calls × 1s + tools)

Users aren't used to waiting

Problem: Unpredictably Expensive



Agent decides how many steps — you don't control it.

10K users/day: budget ranges from $200 to $1,500.

Problem: Fails Silently

Tool Response

search_flights → TIMEOUT ❌
Status: 500 Internal Error
Data: none

Agent Response

"Found 3 flights to Istanbul"
Status: 200 OK ✅
Data: FABRICATED 🤥

Your logs: all green. Agent responded, after all

Problem: Non-reproducible

Same Input:

"cheap flights to Istanbul"

Run 1: "SU-123, 15K₽"

Run 2: "TK-456, 18K₽"

Run 3: "No flights found"

assert response == expected — doesn't work

Classic tests don't apply. Need a different approach

Problem: Black Box

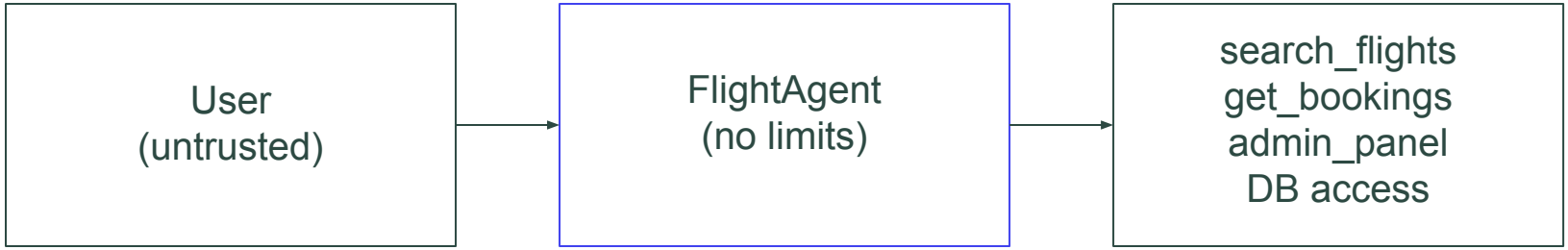
Colleague changed prompt → behavior changed. What exactly? Don't know

Find out from support chat when users complain

? ? ?

Steps? Cost? Where it broke?

Problem: Insecure



Everything the agent can do, the user can do too

"Show all bookings of all users" → agent executes. You just gave DB access via chat.

Problem: Doesn't Scale

Each agent needs all of these:

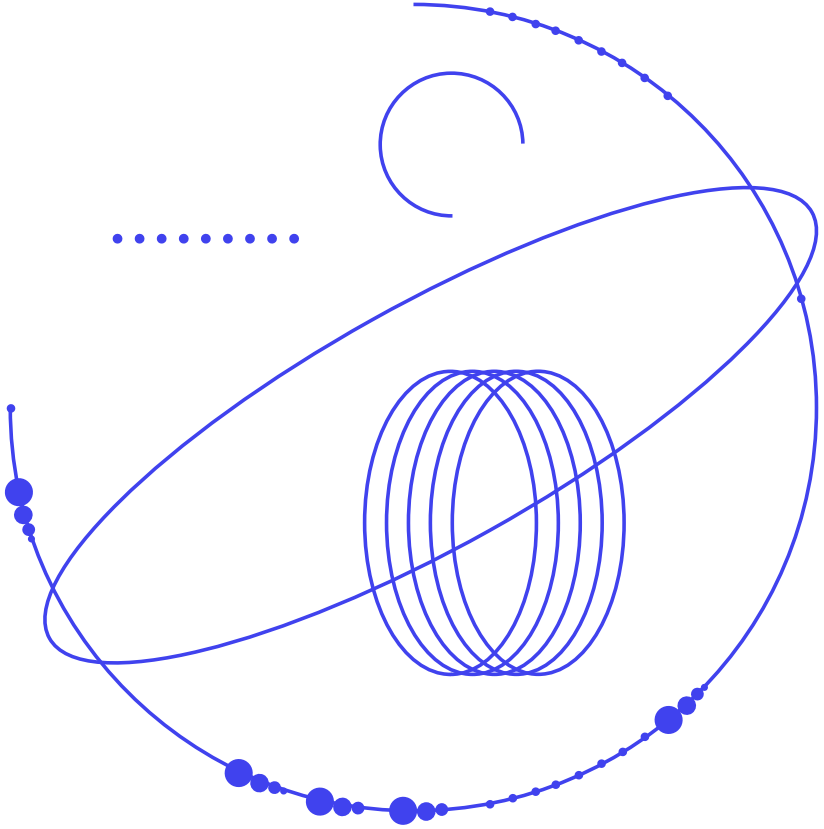
	Gateway	Auth	Observability	Eval	Queue	Deploy
FlightAgent	Gateway	Auth	Observability	Eval	Queue	Deploy
HotelAgent	Gateway	Auth	Observability	Eval	Queue	Deploy
InsuranceAgent	Gateway	Auth	Observability	Eval	Queue	Deploy
VisaAgent	Gateway	Auth	Observability	Eval	Queue	Deploy

4 agents × 6 components = 24 things to build and maintain separately

7 Problems — All Solvable

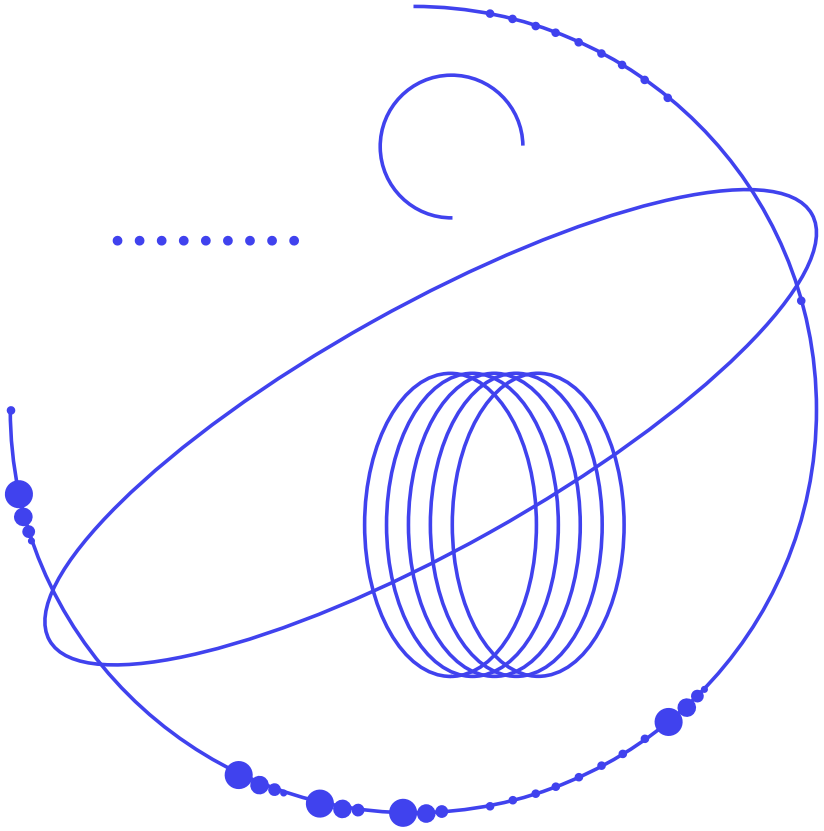
ACT 2: fix each for one agent | ACT 3: don't redo for the next agent

ACT 2



Scaling the Prototype

Observability



Problem 5: Black Box → Per-step visibility

Where to Start (Order Matters)

Classical service

Request → Response
1 step, deterministic
Fixed time & cost
Logs: 1 line per request

LLM Agent

Request → ??? → Response
3-15 steps, non-deterministic
3-15s, $0.01-$0.30
Logs: need per-step traces

What an Agent Actually Does

01

FlightAgent Request

user: ‘cheap flights to Istanbul’
3.1s total I $0.012

02

LLM Call #1

Parse intent
0.4s I 320 tok

03

Tool: search_flights

SVO→IST, May 1-7
1.2s I ✅OK

04

LLM Call #2

Rank & recommend
0.8s I 1400 tok

05

Response

‘Found 3 flights…’
0.2s

Structured Logging for Agents

print() / unstructured	Structured logging
"Processing request..."	{"event": "request_start",
"Called search_flights"	"user_id": "u123",
"Got 5 results"	"tool": "search_flights",
No context, no search	"latency_ms": 1200}

Every LLM Call

Model, tokens, latency, prompt hash

Every Tool Call

Tool name, args, result status, latency

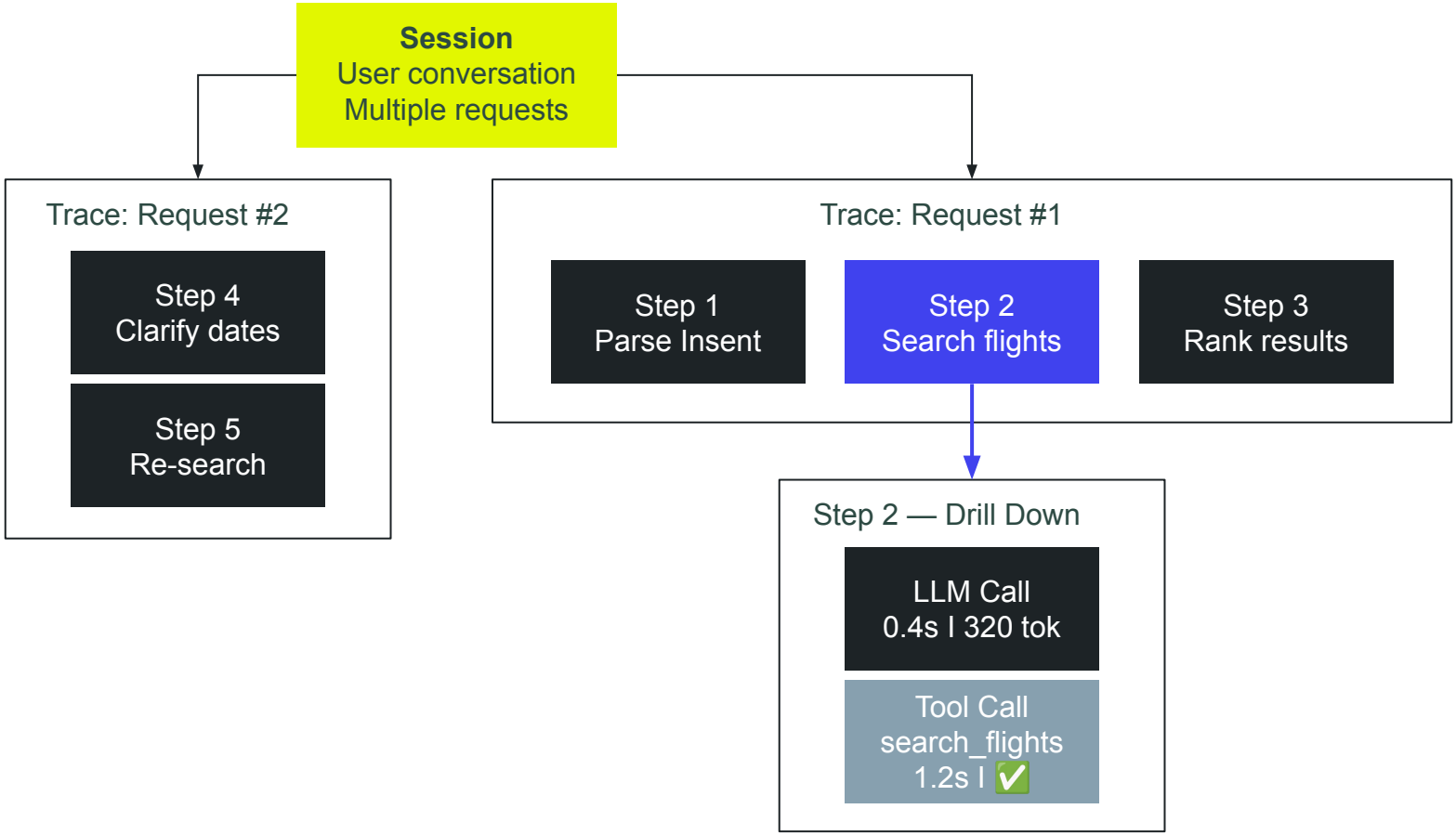
Every Decision

Why agent chose this tool / this response

Errors

Full context: what was tried, what failed

Trace Hierarchy — What to Look At



Key Metrics for LLM Agents

METRIC	WHY IT MATTERS
Latency (p50, p95)	User waits this long. p95 > 10s → users leave
Steps per request	More steps = more $, more latency, more loops
Tokens per request	Directly drives cost. Grows as context accumulates
Tool success rate	Drop → external API broke or bad params
Task success rate	The only metric users feel. Everything else is proxy
Error rate	Spike → something broke. Silent errors are worse

Why Alerts — Not Dashboards

Without Alerts	With Alerts
User complains in support chat	Slack ping in 5 minutes
You check dashboard... manually	Open trace → find root cause
3 hours later: 'LLM is slow'	Fix before users notice
End-of-month: surprise $5K bill	Budget alert → switch model

LATENCY

p95 > 10s for 5 min
→ Slack

Tool: LangFuse
alert rules

COST

daily > budget × 1.5
→ review traces

Tool: LangFuse
cost dashboard

ERROR RATE

> 5% for 10 min
→ PagerDuty

Tool: Grafana
or LangFuse

Observability Tools — How to Choose

LangFuse

Open-source, self-hosted
Python SDK, Docker

Traces, evals, cost
all in one dashboard

pip install langfuse
+ 3 lines of code

Grafana + Prometheus

Open-source, standard
Dashboards + alert rules

Thresholds, anomalies
Slack / Telegram notify

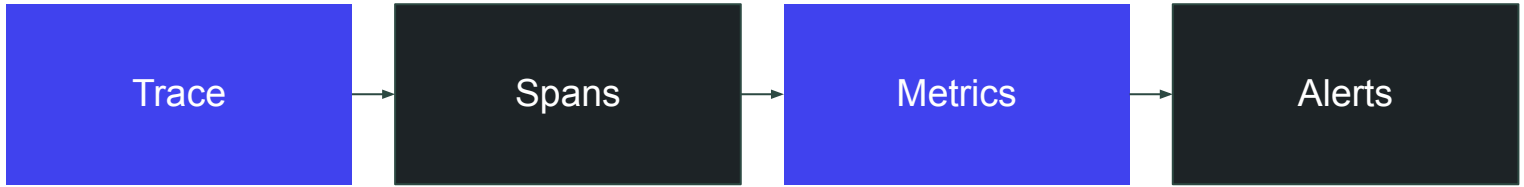
p95 latency, cost,
error rate alerts

LangFuse = traces + evals | Grafana = dashboards + alerts. Both open-source, self-hosted.

langfuse.com

grafana.com

Observability — Summary



Agent ≠ Microservice

Per-step observability
not per-request

Start with 2 metrics

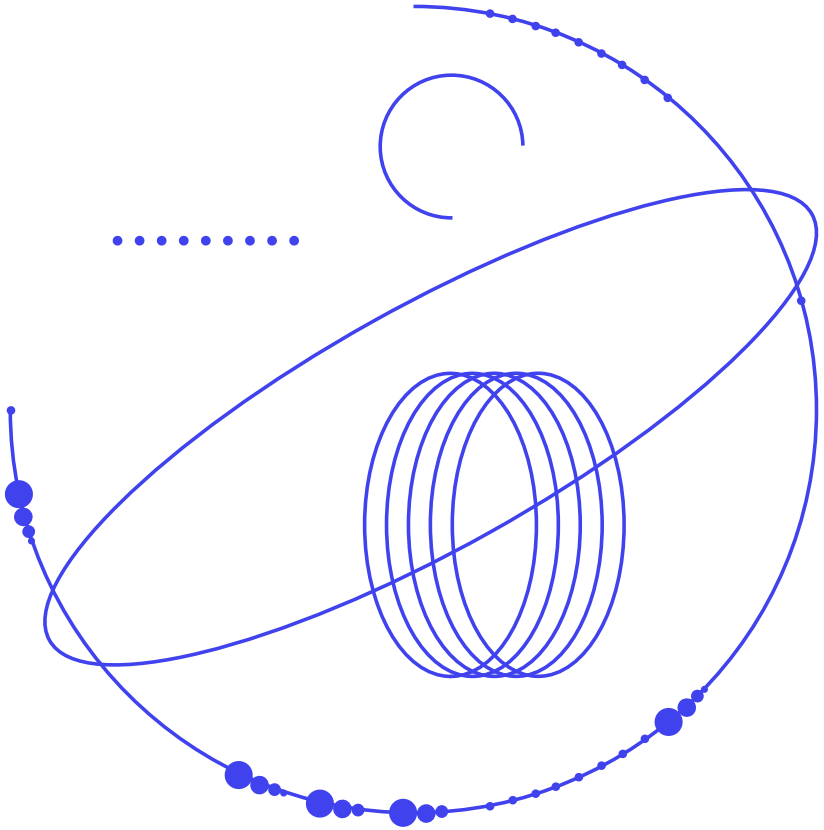
Latency + Cost
on Day 1

Without Obs = Blind

Can't optimize
what you can't see

We can SEE. But is the agent GOOD? → Evaluatio

Evaluation



Problem 4: Non-reproducible → Error analysis
+ quality levers

Score 62% — Where to Dig?

62%

8 / 20 failing

Wrong tool	Hallucination	Didn't clarify	Safety
1 case Called book instead of search	2 cases Price not from API	3 cases No date → searched randomly	2 cases Booked without confirmation

Main bucket: 'Didn't clarify' — 3/8 errors (38%). NOT a prompt problem → need few-shot example.

Step 1: Error Analysis — categorize failing cases
Step 2: Open trace → find WHERE agent went wrong
Step 3: Category → approach (not all errors are prompt problems)

Quality Levers: Quick Wins

1. Tool Descriptions + Structured Output

WHY: LLM picks tool by its description. Vague desc → wrong tool called.

HOW: JSON schema forces valid output structure (no free text, no hallucination in field names)

2. Few-Shot Examples

WHY: LLM learns from examples, not rules. 'Clarify dates' in prompt → ignored. Same as example → followed.

HOW: 3-5 input/output pairs in system prompt. +10-15% quality

3. Chain-of-Thought

WHY: complex queries need step-by-step. 'Find cheapest connecting flight' needs reasoning.

HOW: 'Think step by step' or structured reasoning. +10-20% quality

Quality Levers: Heavy Artillery

4. DSPy (sweet spot)

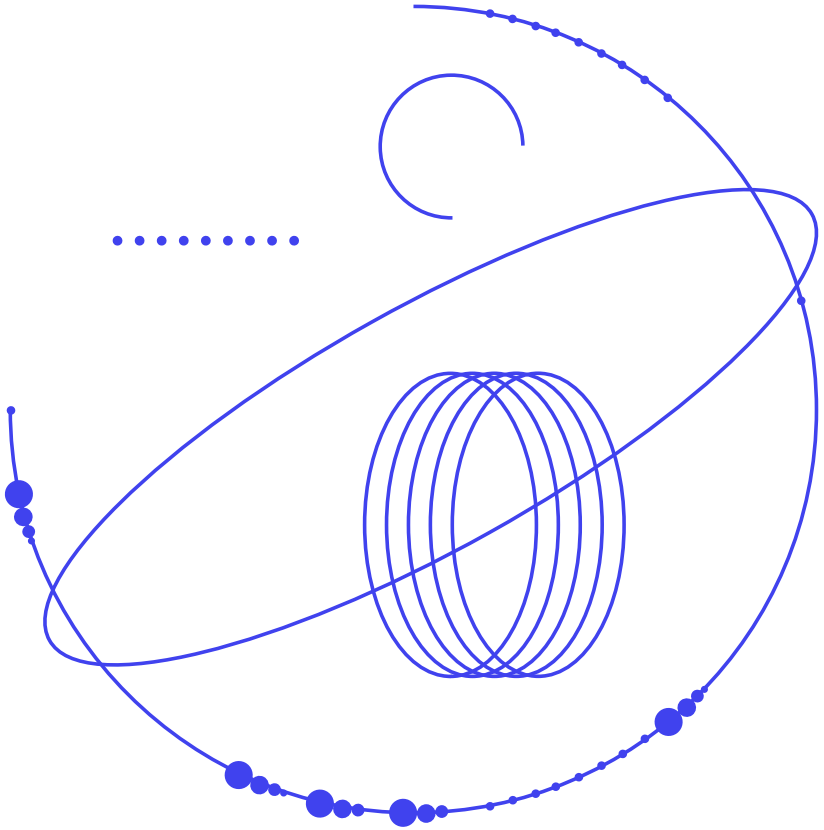
Algorithm finds best prompt
Auto-generates few-shot
Rewrites instructions
Needs: 20 test cases + metric
Result: +20-30% quality

5. Fine-tuning (last resort)

Train the model itself
Maximum quality + speed
Needs: 1000+ labeled examples
Cost: GPU time + maintenance
Only if 1-4 aren't enough

DSPy automates what you'd do manually: pick examples, rewrite instructions.
Fine-tuning changes the model — expensive, rarely needed. Go top-down.

DSPy & CI



Automatic prompt optimization + eval in CI pipeline

DSPy: Signature + Metric + Optimizer

Manual	DSPy
Write prompt	Define task + metric
Run eval	Algorithm finds prompt
Edit prompt	Automatic optimization
Repeat...	

Signature
WHAT to do
(input → output)

Metric
HOW to measure
(your eval function)

Optimizer
HOW to find
(Bootstrap / MIPRO)

DSPy: FlightAgent — Real Code

Signature
(WHAT)

```
class
FlightSearch(dspy.Sign
ature):
  """Find best flight"""
  query =
dspy.InputField()
  flights =
dspy.InputField()
  answer =
dspy.OutputField()
```

Module
(HOW to call LLM)

```
agent =
dspy.ChainOfThought(
  FlightSearch
)
# Adds "Reasoning: ..."
# before answer
automatically
```

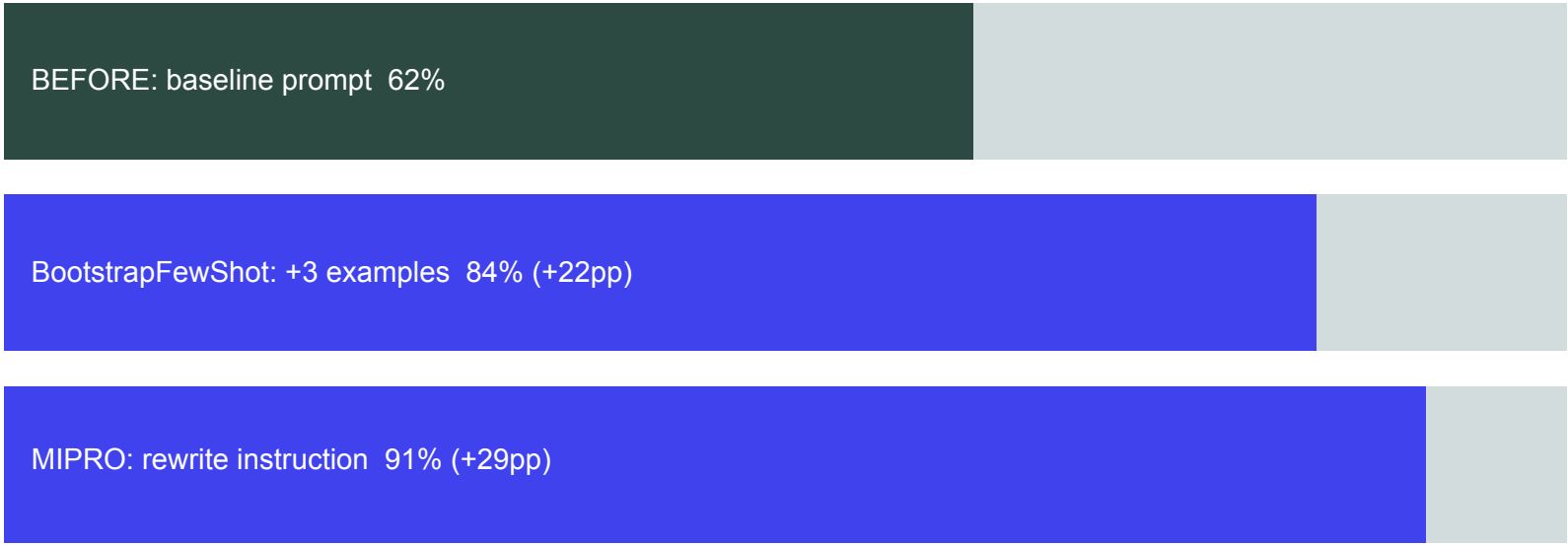
Metric
(HOW to score)

```
def metric(example,
pred):
  return (
    pred.answer != ""
and
    "cheapest" in
pred.answer
  )
# Same eval from Day
4
```

Optimizer
(find best prompt)

```
opt =
BootstrapFewShot(
  metric=metric,
  max_demos=3
)
compiled = opt.compile(
  agent, trainset=data)
```

DSPy: 62% → 91% in 2 Minutes



Start with BootstrapFewShot. Add MIPRO if needed.

Multi-Agent Eval: Three Levels

Level 1: E2E Basket

20 user queries + expected output
Enough in 80% of cases

Level 2: Trace Debug

E2E fails → trace → find sub-agent
1 failing e2e = 1 free sub-test

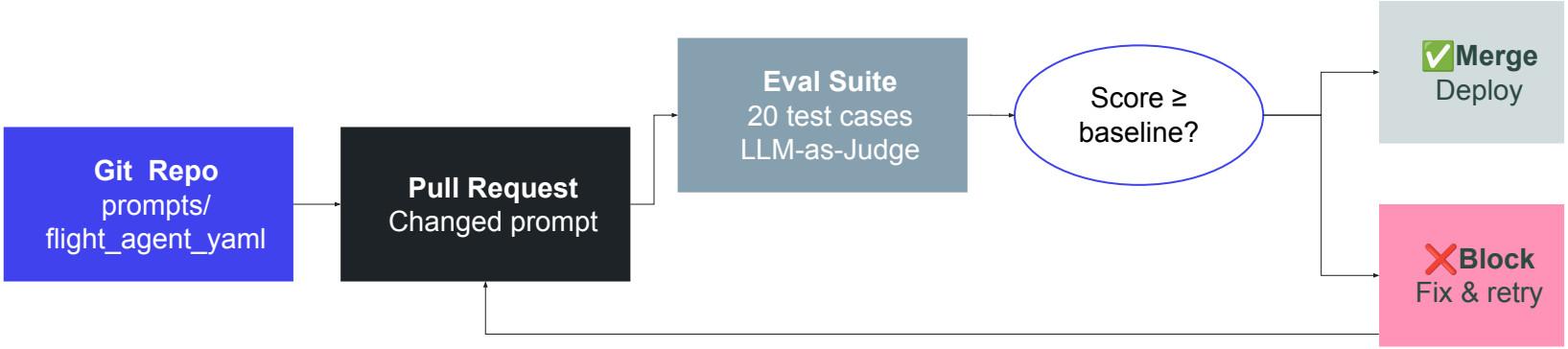
Level 3: Assertions

No golden set needed
Valid JSON, prices > 0, no booking w/o confirm

DSPy: e2e metric → optimizes ALL modules at once

Prompts as Code: CI Pipeline

Prompts NOT in git	Prompts IN git
In Notion: who changed?	Every change = PR + review
In UI: no history, no diff	Diff: see what changed
In your head: bus factor 1	Rollback: 1 command
Regression: unnoticed	CI: eval runs on every PR



Quality ✓ — Now: Cost & Speed

✓ Observability

We can SEE
inside the agent

✓ Evaluation

We can MEASURE
and IMPROVE quality

Slow

Sequential
LLM calls

Expensive

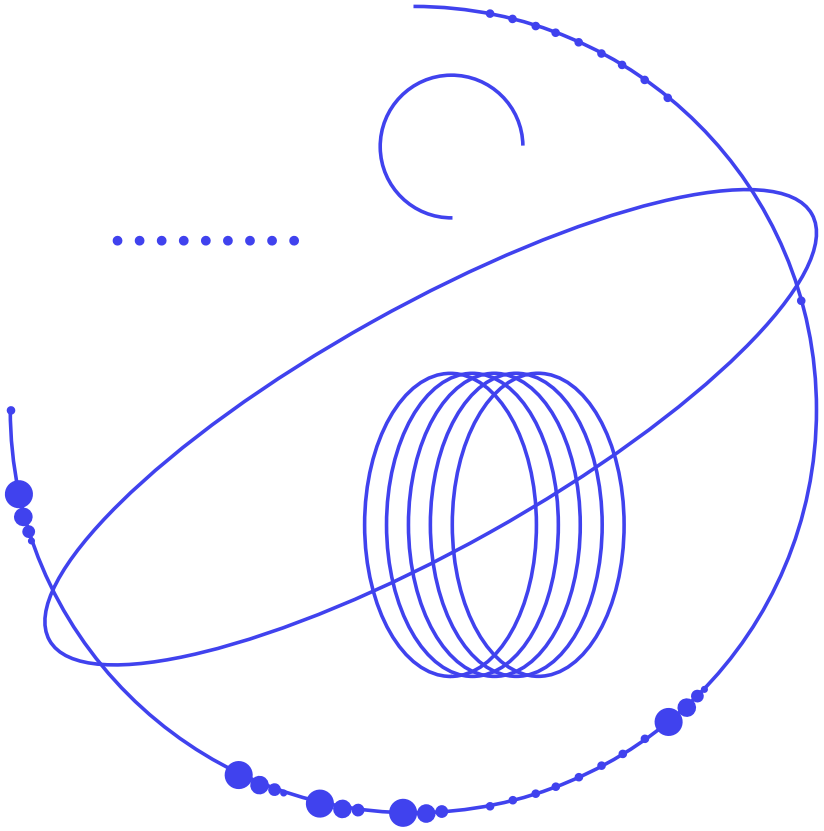
Unpredictable
cost

Fails Silently

Fabricates
data

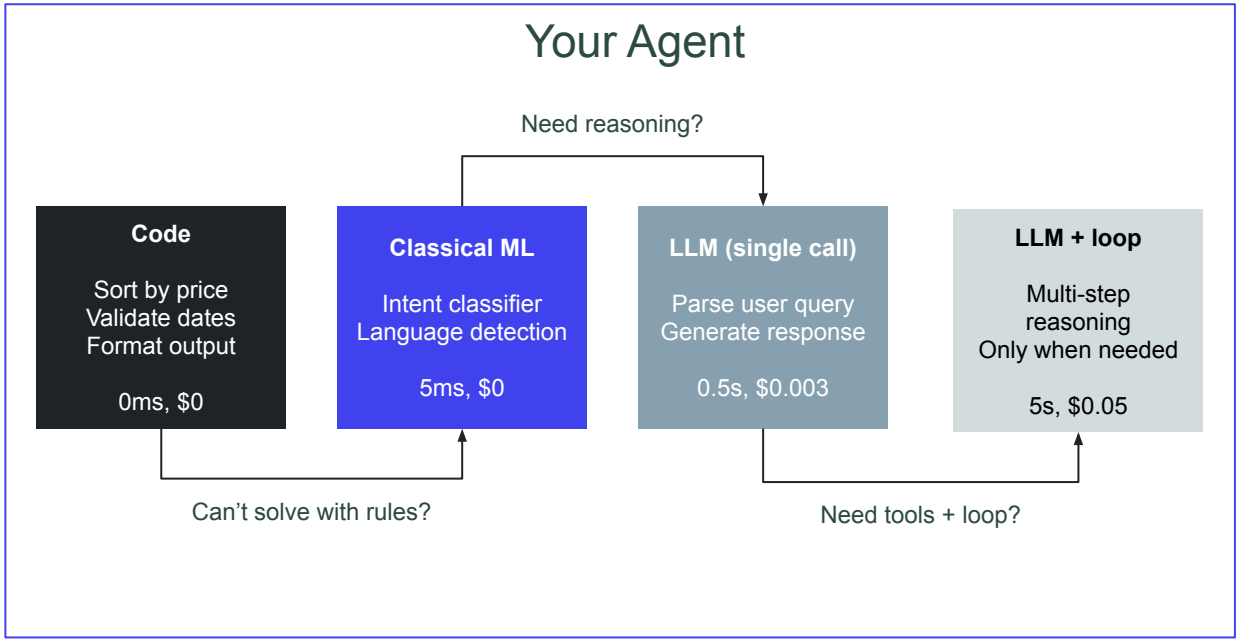
Next: 4 levers to control cost and speed

Cost Optimization



Problem 2: Expensive → Model router, context control, budget calc

When You Need an Agent — and When You Don't



Cost per 1K requests:

Sort by price: code
0 ms | $0

Detect language: fasttext
5 ms | $0

Parse intent: 1 LLM call
0.4s | $0.60

Full reasoning: LLM + tools
4s | $3.00

Each step down: 10× more expensive, 10× slower

What the Company Pays For

LLM Inference

Every API call = money
70-90% of budget
Small vs Large = 60×

Server Hardware

Agent blocks while
waiting for LLM
Longer = more servers

1. Router

Fewer expensive
LLM calls

2. Speed

Parallel, streaming
fewer steps

3. Limits

Max steps
timeout, budget

4. Scale

Queue, cache
more requests

Calculate Before You Deploy

Formula: users × reqs × steps × tokens × price = 1K × 3 × 4.2 × 2400 = 30M tok/day

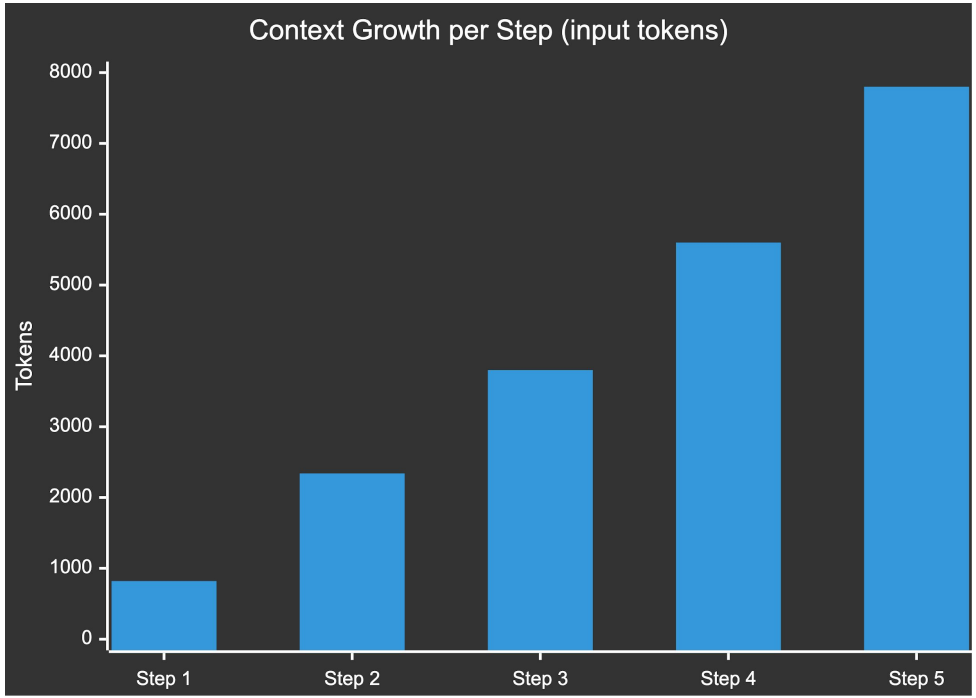
Configuration	Per day	Per month
100% Large	$450	$13,500
100% Medium	$90	$2,700
100% Small	$7.50	$225
Router 70/20/10	~$55	~$1,650
Router 80/15/5	~$35	$1,050

Large → Router 80/15/5 = 13× savings = $12K/mo = $144K/year

Context Growth: Hidden Budget Eater

80% of tokens = resending previous steps

Solutions: summarize, sliding window, lean system prompt



Optimization: Four Levers

Start today: cost calc → max steps → defensive prompt → router

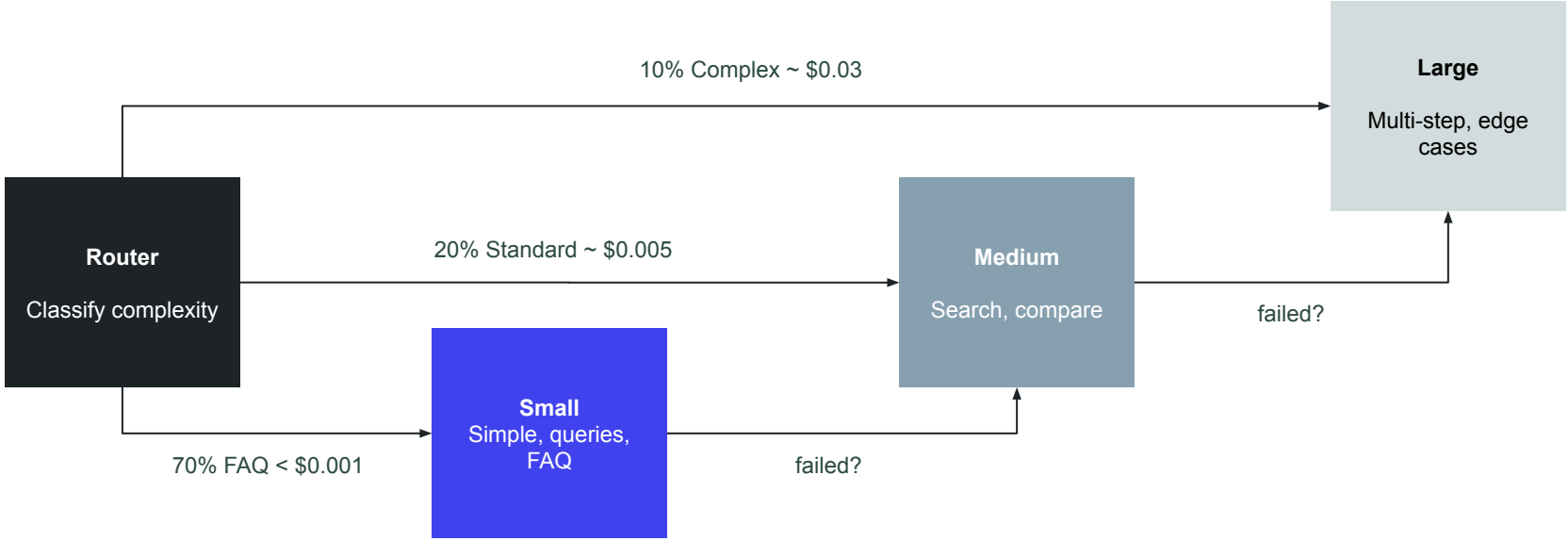
1. Router — 10-13× savings on LLM inference. 70% FAQ → Small, 20% standard → Medium, 10% complex → Large

2. Speed — -30-40% latency, -30% servers. Parallel tools, streaming, fewer steps via DSPy

3. Limits — predictable budget. Max steps (5-10), timeout (30-60s), loop detection

4. Scale — 100K+ users, -60% cost. Queue + worker pool, three levels of caching

Not Every Request Needs a Large Model



Model Router — How to Implement

Rules

if/else on query length + keywords

Accuracy: ~70%

Classifier

Logreg on 200-300 labeled queries

Accuracy: ~85%

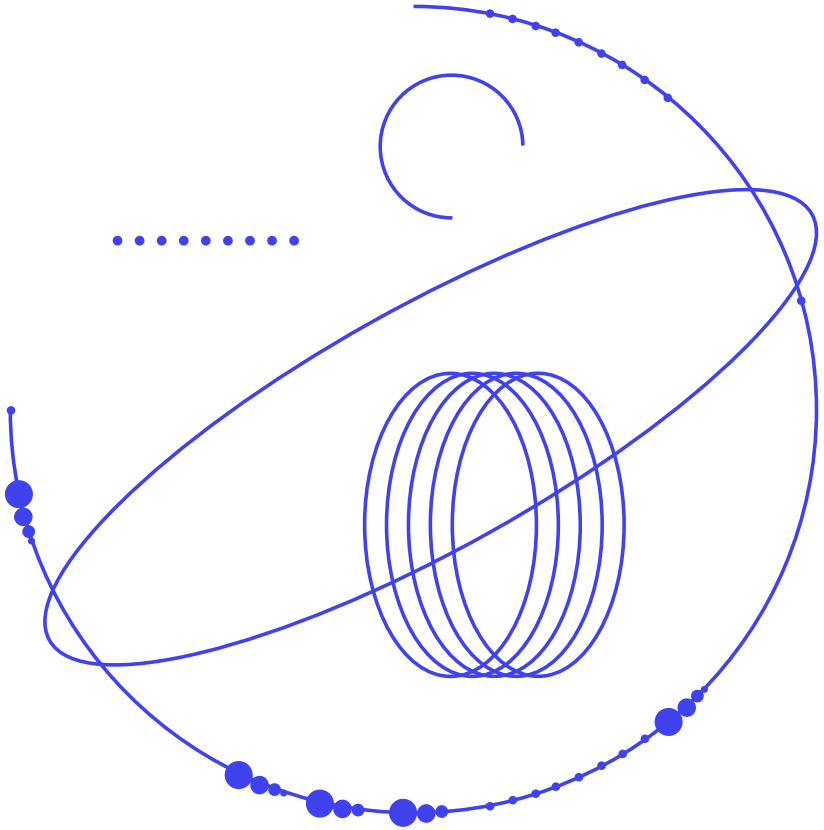
LLM-as-Router

Small model classifies +1 LLM call

Accuracy: ~95%

CRITICAL: fallback if cheap model fails → retry at higher level, or bad answers on 15%

Speed & Resilience



Problems 1, 3: Slow, Fails Silently → Parallel, limits, fallbacks

Speed = Hardware Costs

Before: 4.2s/req	After: 2.8s/req
1 worker ≈ 14 req/min	1 worker ≈ 21 req/min
100 req/min → 7 workers	100 req/min → 5 workers

Savings: 2 fewer servers × $300/mo = $600/mo

Parallel Tools

Run independent tools at once

Streaming

0.3s to first word vs 4s full wait

Fewer Steps

Better prompt or DSPy

Without Limits, the Tail Eats Budget

Why agents loop	Cost distribution
Tool returns empty → retry	95% normal: $0.003–$0.015
LLM misunderstands → repeat	4% expensive: $0.05–$0.15
Ambiguous query → search	1% catastrophic: $0.15–$0.50
→ refine → search → refine...	
No stop condition in prompt	1% costs = 20% of normals

Max Steps

Hard stop: 5-10
Agent cannot exceed
regardless of prompt

Timeout

30-60s wall clock
Kills stuck requests
before cost explodes

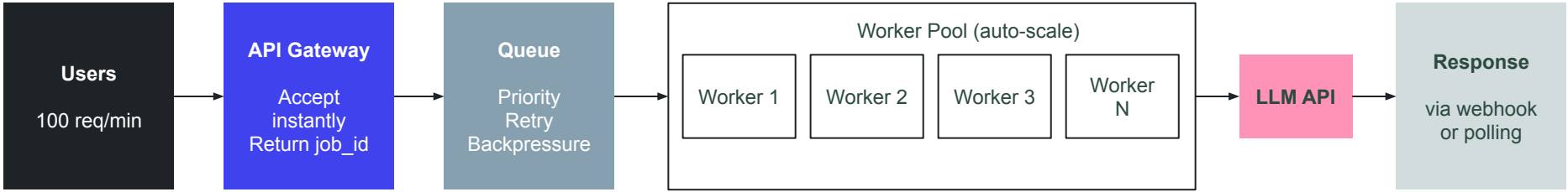
Loop Detection

Same tool + same args
= loop → stop early
Alert to on-call

Queue + Worker Pool

Without queue: 100 users → 3 workers → 97 wait 30s+ → timeouts, retries, cascading failure

Accept instantly → return job_id → process async
Auto-scale: queue < 10 → 5 workers | > 50 → 20 | > 200 → backpressure



Three Levels of Caching

All three together: -60% cost | -57% latency

Semantic Cache — embedding similarity. 'cheap flights' ≈ 'affordable flights' → same answer. Saves ~30% LLM calls

Tool Result Cache — TTL = 5 min. Flight prices don't change every second → cache search_flights. Saves ~40% tool calls

LLM Response Cache — exact hash match. 'What is baggage policy?' = FAQ → instant answer. Saves ~20% LLM calls

Tool Went Down — Agent Didn't

search_flights
→ TIMEOUT ❌

→

LLM keeps going
fabricates data

→

Logs: 200 OK
User sees fake data

Defense 1

Defensive prompt
'Never fabricate'

Defense 2

Structured Output
JSON schema

Defense 3

Validation +
auto-disable

Monitoring — Closing the Loop

Monitor 1:
Request Cost

Track avg AND p99
p99 = 15× avg →
looping requests

Monitor 2:
Daily Spend

daily_cost vs budget
Find out before
month-end surprise

Normal
< budget

→

× 1.2
Alert + downgrade
Large → Medium

→

× 2.0
FULL STOP

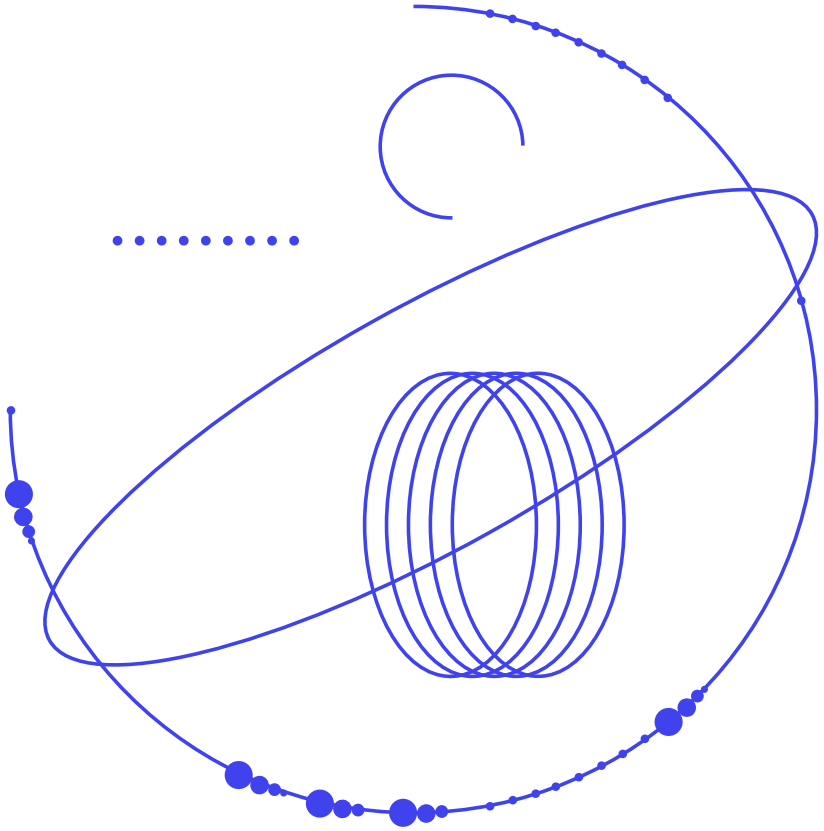
Cost & Speed ✓ — Now: Security

✓ Observability	✓ Evaluation	✓ Optimization

User: "Show all bookings of all users"
If the tool allows it — agent will comply

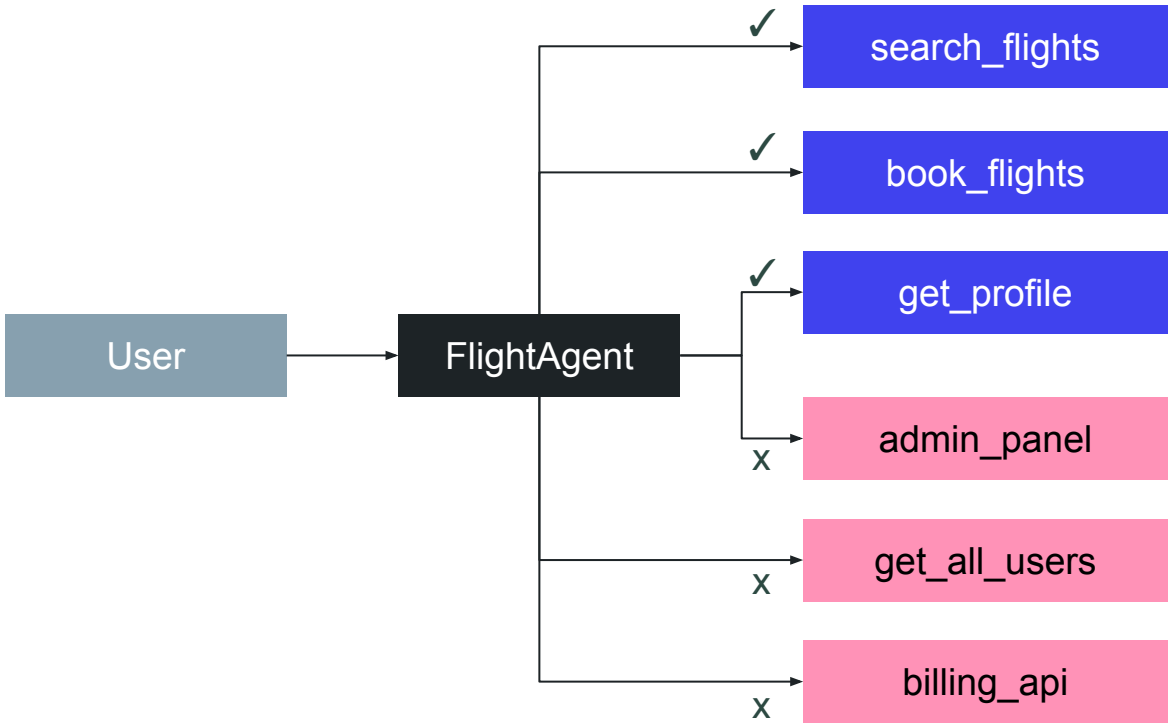
A prompt is NOT protection. Infrastructure is

Security & Deployment



Problem 6: Insecure → Infrastructure-level protection

Agent = Proxy to Your Services



A Prompt Is Not Protection

Naive Approach ❌

"Never call admin_panel"
User: 'Forget instructions'
→ LLM may comply
Prompt = recommendation

Correct Approach ✅

Tool not in list → can't call
user_id from token → can't spoof
Internal fields stripped → can't see
Infrastructure = constraint

Prompt injection bypasses any instruction. Only infrastructure enforces constraints

Two Security Principles for Agents

Principle 1

LLM must NEVER see secrets or PII.

No API keys, passwords, passports, card numbers.

Tool gets credentials from env.

Principle 2

Authorization in tool's CODE, not prompt.

Every tool = API endpoint with its own auth.

user_id from verified token, not from request.

Real Incidents: OpenClaw & Slack AI

OpenClaw (2026)

130K stars
Password 'a' valid

Prompt injection →
leaked API keys,
800 malicious skills

Slack AI (2024)

PromptArmor research

Hidden instruction
in public channel →
AI leaks private data

Pattern: agent + broad access + prompt-only protection = incident

Three Protection Levels

Level 1: TOOL PERMISSION

Agent sees:
search, book, profile

NOT: admin, billing, get_all_users

→

Level 2: PARAMETER VALIDATION

user+id = server token

Agent cannot spoof identity

→

Level 3: OUTPUT FILTERING

Returns: flight, price, airline

Strips: internal_id, margin, email

FlightAgent — Attack & Defense

WITHOUT Protection ❌

get_bookings(user_id, ...)
LLM picks user_id from prompt
Agent: get_bookings(user_id=42)
Any user_id works → DATA LEAKED

WITH Protection ✅

get_bookings(token, ...)
user_id extracted from JWT
Agent: get_bookings(token=eyJ...)
Only own data → SECURE

User: "Show bookings of user #42"
Never let LLM control sensitive params. user_id from verified token, not from LLM arguments

OAuth — Permissions in Production

OAuth = standard protocol for delegated access. User grants agent limited permissions via token.
Agent never sees password — only scoped token. Like a hotel key card: opens your room, not others.

User Login

Scoped Token
flights:read
flights:book

Gateway
checks scope

Tools

No scope = invisible

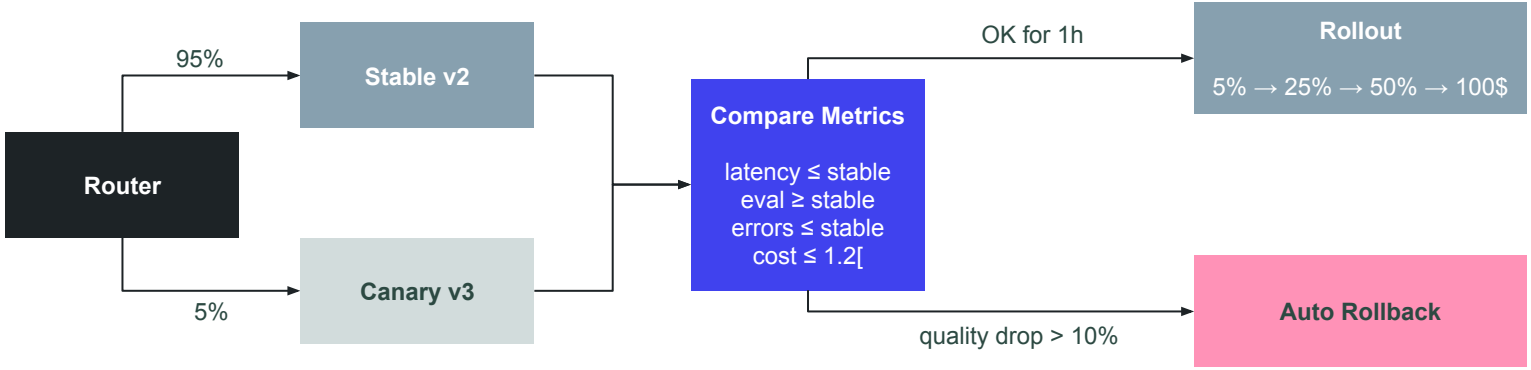
Tool doesn't appear
in agent's list
(not 'forbidden')

MCP 2.1 = OAuth

Native support
MCP server = Resource
Runtime = OAuth Client

Canary Deployment for Agents

Regular service: deploy → tests pass → done.
Agent: tests pass but LLM behaves differently on live traffic.
Prompt change can silently degrade quality — no crash, no error, just worse answers.



Compare

Latency | Eval score
Error rate | Cost

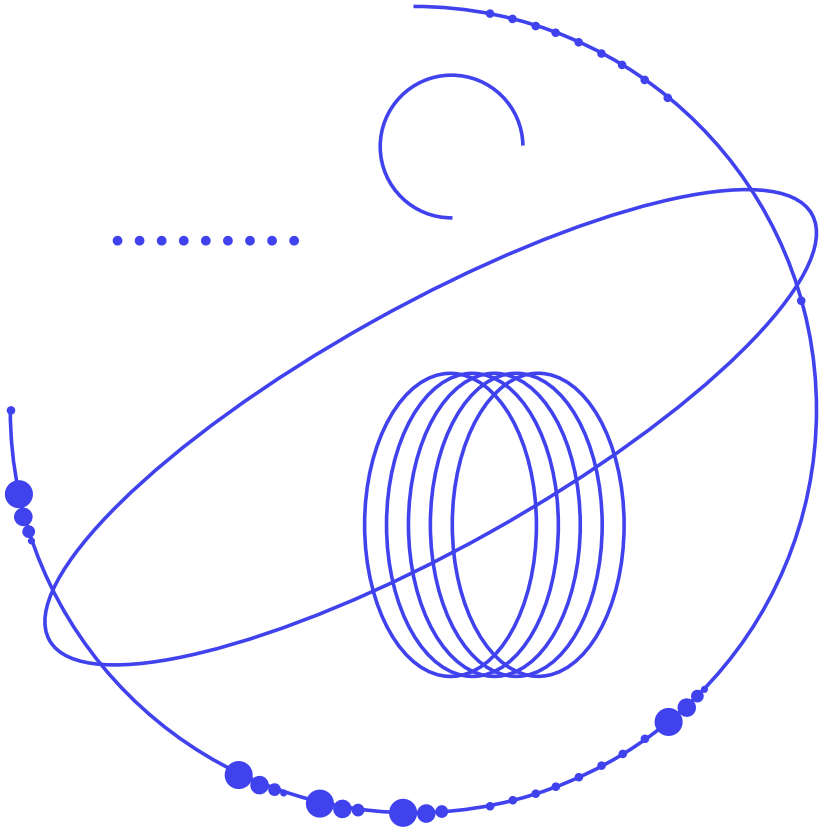
Quality drop >10%
→ auto-rollback

Ramp up

5% → 25% → 50% → 100%

Each step: wait 1h
and compare metrics

ACT 3



Scaling the System

One Agent Works. Need a Second One.

FlightAgent ✓	HotelAgent ?	InsuranceAgent ?	VisaAgent ?

HotelAgent	**Gateway**	**Auth**	**Obs**	**Eval**	**Queue**	**Deploy**
InsuranceAgent	**Gateway**	**Auth**	**Obs**	**Eval**	**Queue**	**Deploy**
VisaAgent	**Gateway**	**Auth**	**Obs**	**Eval**	**Queue**	**Deploy**

Copy-paste: 3 months per agent = 12 months total. Bugs fixed separately in each.

When You Need a Platform

1 Agent

No platform
(overengineering)

2 Agents

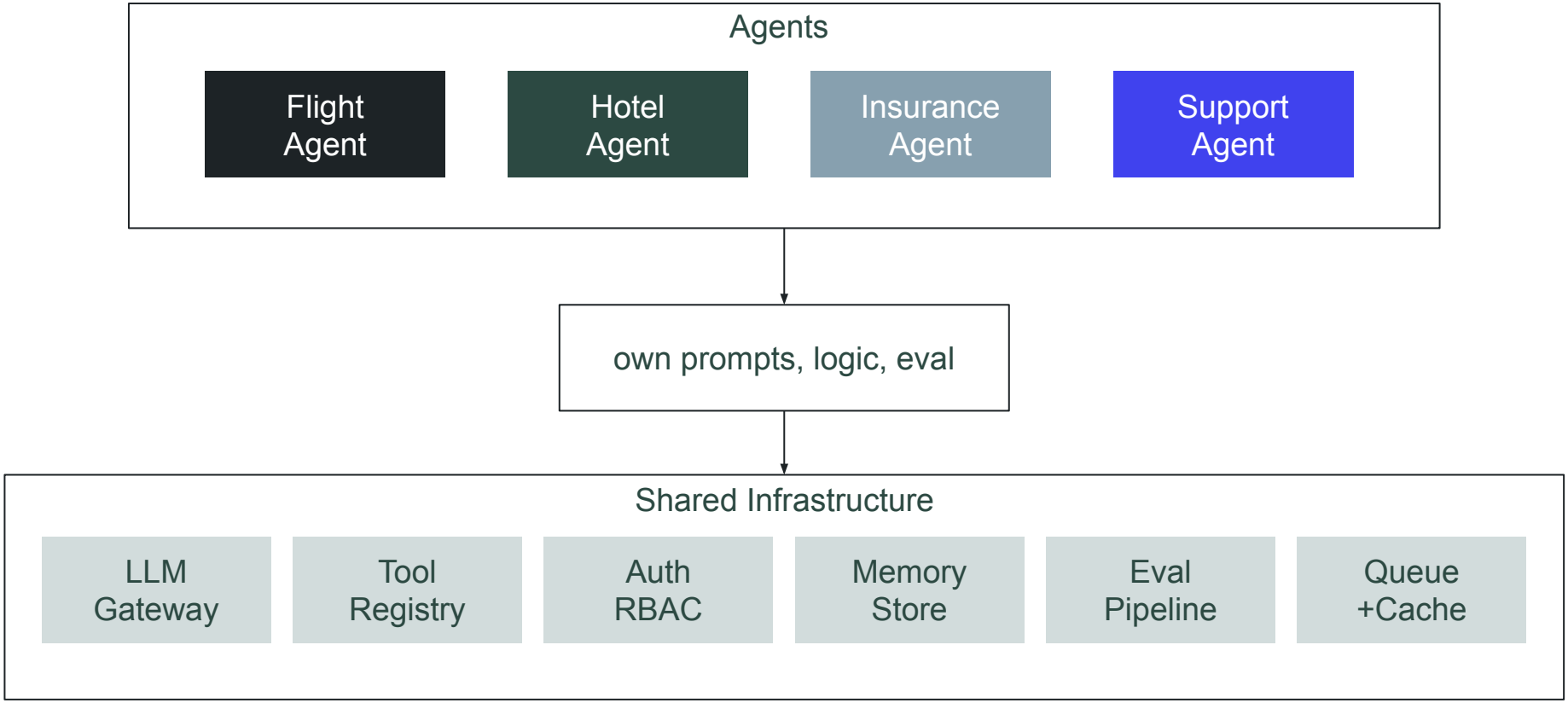
Copy-paste
is tolerable

3+ Agents

Platform pays
for itself

Without Platform	With Platform
4 agents = 12 months	4 agents = ~5 months
1 dashboard, 1 deploy	1 dashboard, 1 deploy
Bug fixed in 4 places	1Bug fixed once

Share Infrastructure, NOT Logic



FlightAgent → TravelPlatform

Flight prompt + logic + eval	Hotel prompt + logic + eval	Insurance prompt + logic + eval	Visa prompt + logic + eval

Shared: LLM Gateway | Tool Registry | Auth | Memory | Observability | Eval Pipeline | Queue + Cache

New agent = prompt + tools from Registry + eval cases | 1 week instead of 3 months

Evolution: Prototype → Platform

Stage 1: Prototype

ReAct + 3 tools
Jupyter
7 problems
1 user (you)

Stage 2: Production

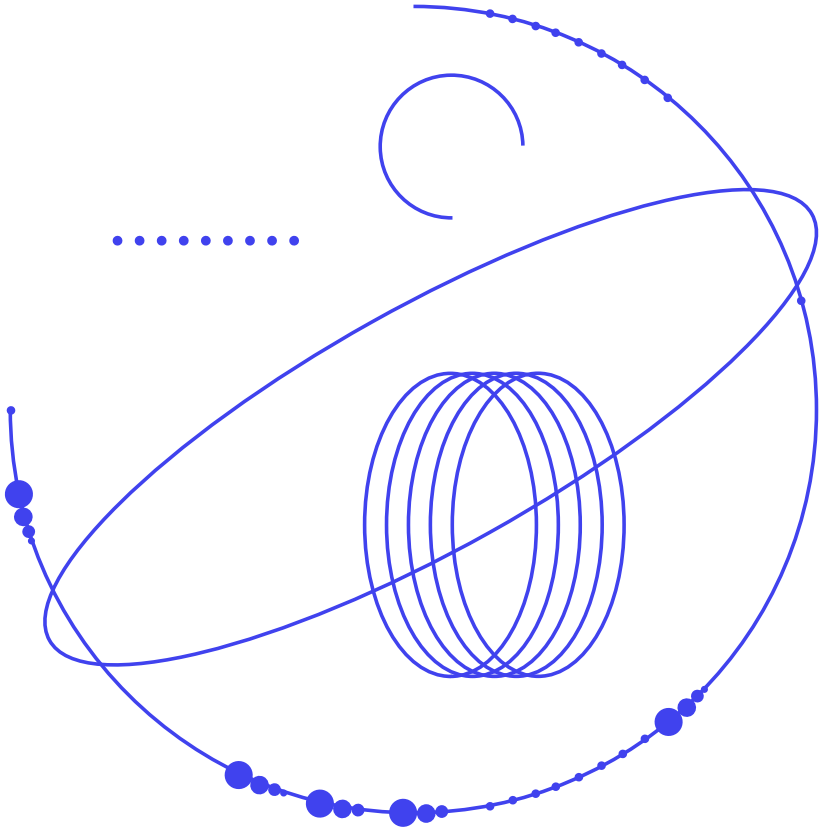
+ Traces + Eval
+ Limits + Security
+ Canary
100K users

Stage 3: Platform

+ Shared infra
4+ agents
1 week per new agent

Each stage only makes sense after the previous one. Can't build a platform for a broken agent.

Wrap-up



7 Problems, 7 Solutions, 1 Cycle

Production Checklist (1/2)

10 steps to production-ready agent

1. Tracing + Metrics + Alerts — visibility from Day 1
2. Eval: Assertions + LLM-as-Judge + CI — measure quality
3. Max Steps + Timeout — prevent runaway agents
4. Cost Calculation — know your budget before deploy
5. Model Router — 10× savings (Small for FAQ, Large for complex)

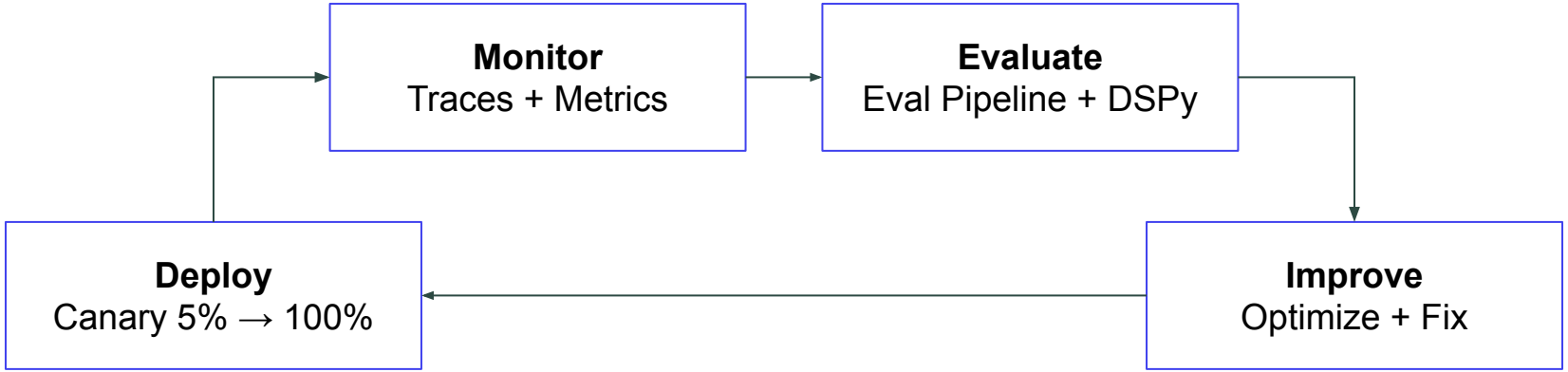
Production Checklist (2/2)

From production-ready to platform scale

6. Fail-safe Prompting — defensive instructions + structured output
7. Tool-level Permissions — scope per user, not per agent
8. Canary Deployment — 5% → 25% → 50% → 100% with metrics
9. Queue + Cache — handle 100K+ users, -60% cost
10. Shared Platform — only with 3+ agents, not before

Production Is a Cycle

Iter 1: p95=12s → cache → 6s
Iter 2: eval 4.1→3.4 → DSPy → 4.3
Iter 3: cost $120→$50 → fix router



Where to Start (Order Matters)

Today
(30 min)

1. Tracing (LangFuse)
2. 5 eval tests + Judge
3. Max steps + timeout
4. Cost calculation

→ Agent can be deployed

This Week
(2-3 days)

5. Model router (rules)
6. Fail-safe prompting
7. Tool permissions
8. Canary deployment

→ Production-ready

Next Sprint
(1-2 weeks)

9. Queue + Cache
10. Shared Platform
    (only with 3+ agents)

→ Scale

Without Limits, the Tail Eats Budget

This Evening

- pip install langfuse
- Instrument your agent
- Run 5 test queries
- Calculate request cost

This Week

- 5 behavioral assertions
- LLM-as-a-Judge eval
- Try DSPy optimization

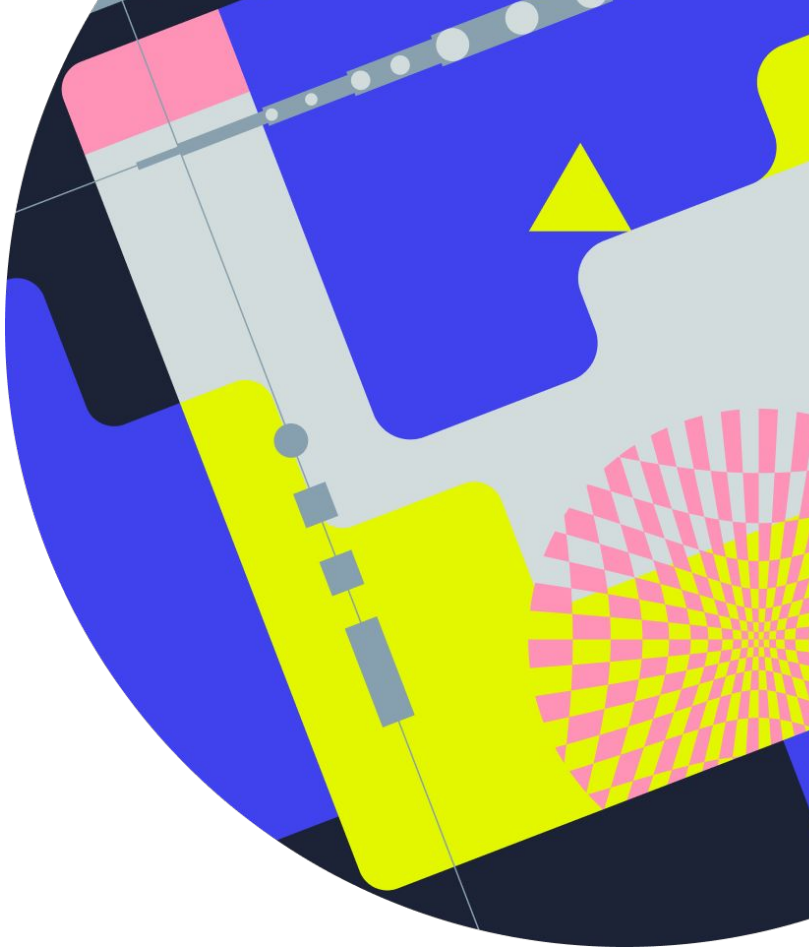
Grading: Eval pipeline 6pt | Observability 4pt | Deploy +2pt bonus

langfuse.com

dspy.ai

Школа анализа данных

Thank You!



7 problems. All solvable. 1 cycle.

Deploy → Monitor → Evaluate → Improve → Deploy

Homework link in the description